Full Sync 완전 검증 분석

개념적 배경

Full Sync는 Ethereum의 가장 보수적이고 완전한 동기화 방식으로, 제네시스 블록부터 최신 블록까지 모든 블록을 순차적으로 다운로드하고 실행하여 전체 상태를 재구성합니다. 이는 가장 높은 수준의 보안과 검증을 제공하지만, 시간과 리소스가 많이 소요되는 방식입니다.

설계 필요성

1. **완전한 검증**: 모든 트랜잭션을 실행하여 상태 변화를 직접 검증
2. **최고 보안 수준**: 제3자에 대한 신뢰 없이 독립적으로 체인 검증
3. **히스토리 보존**: 전체 블록체인 히스토리와 상태 변화 추적 가능
4. **네트워크 기여**: 완전한 노드로서 네트워크에 최대한 기여

다른 접근법과의 비교

- **Fast Sync**: 헤더 체인 + 상태 다운로드, 빠르지만 일부 검증 생략
- **Snap Sync**: 스냅샷 기반 동기화, 매우 빠르지만 히스토리 제한
- **Light Sync**: 헤더만 동기화, 빠르지만 기능 제한
- **Full Sync**: 모든 블록 실행, 느리지만 완전한 검증과 히스토리

구현 분석

핵심 데이터 구조

BlockChain 구조체

```go
type BlockChain struct {
chainConfig *params.ChainConfig // 체인 설정
db ethdb.Database // 블록체인 데이터베이스
// 상태 관리
stateCache state.Database // 상태 데이터베이스
triedb *triedb.Database // 트라이 데이터베이스
// 블록 처리
processor Processor // 상태 프로세서
validator Validator // 블록 검증자
// 체인 헤드 관리
currentBlock atomic.Pointer[types.Header] // 현재 블록
currentSnapBlock atomic.Pointer[types.Header] // 현재 스냅 블록
currentFinalizedBlock atomic.Pointer[types.Header] // 최종 확정 블록
// 캐시
blockCache *lru.Cache[common.Hash, *types.Block]
receiptCache *lru.Cache[common.Hash, []*types.Receipt]
// 가비지 컬렉션
triegc *prque.Prque[int64, common.Hash] // 트라이 GC 큐
gcproc time.Duration // GC 처리 시간
lastWrite  uint64  // 마지막 쓰기 블록 번호
}
```

StateProcessor 구조체

```
type StateProcessor struct {
config *params.ChainConfig // 체인 설정
chain *HeaderChain // 헤더 체인
}

// ProcessResult는 블록 처리 결과를 담는 구조체
type ProcessResult struct {
Receipts types.Receipts // 트랜잭션 영수증
Requests [][]byte  // 시스템 요청 (Prague 포크 이후)
Logs []*types.Log // 이벤트 로그
GasUsed  uint64  // 사용된 가스
}
```

BlockValidator 구조체

```
type BlockValidator struct {
config *params.ChainConfig // 체인 설정
bc *BlockChain // 블록체인 참조
}
```

주요 알고리즘

1. 블록 삽입 알고리즘

Full Sync의 핵심은 모든 블록을 순차적으로 삽입하고 실행하는 것입니다:

```
// InsertChain은 블록 체인을 삽입하는 메인 함수
func (bc *BlockChain) InsertChain(chain types.Blocks) (int, error) {
// 체인 연속성 검증
for i := 1; i < len(chain); i++ {
block, prev := chain[i], chain[i-1]
if block.NumberU64() != prev.NumberU64()+1 ||
block.ParentHash() != prev.Hash() {
return i, fmt.Errorf("non contiguous block insert")
}
}
// 각 블록을 순차적으로 처리
for i, block := range chain {
// 블록 검증
if err := bc.validator.ValidateBody(block); err != nil {
return i, err
}
// 블록 실행
statedb, err := bc.StateAt(block.ParentHash())
if err != nil {
return i, err
}
// 트랜잭션 실행 및 상태 업데이트
```

```
result, err := bc.processor.Process(block, statedb, bc.vmConfig)

if err != nil {

return i, err

}

// 상태 검증

if err := bc.validator.ValidateState(block, statedb, result, false); err != ni

return i, err

}

// 블록과 상태를 데이터베이스에 저장

if err := bc.writeBlockWithState(block, result.Receipts, statedb); err != nil

return i, err

}

}

return len(chain), nil

}
```

2. 트랜잭션 실행 알고리즘

각 블록의 모든 트랜잭션을 순차적으로 실행합니다:

```
// Process는 블록의 모든 트랜잭션을 실행

func (p *StateProcessor) Process(block *types.Block, statedb *state.StateDB,

cfg vm.Config) (*ProcessResult, error) {

var (

receipts types.Receipts
```

```
usedGas = new(uint64)
header = block.Header()
allLogs []*types.Log
gp = new(GasPool).AddGas(block.GasLimit())
)
// EVM 컨텍스트 생성
context := NewEVMBlockContext(header, p.chain, nil)
evm := vm.NewEVM(context, statedb, p.config, cfg)
// 각 트랜잭션 실행
for  i, tx := range  block.Transactions() {
// 트랜잭션을 메시지로 변환
msg, err := TransactionToMessage(tx, signer, header.BaseFee)
if  err != nil {
return  nil, fmt.Errorf("could not apply tx %d: %w", i, err)
}
// 트랜잭션 컨텍스트 설정
statedb.SetTxContext(tx.Hash(), i)
// 트랜잭션 실행
receipt, err := ApplyTransactionWithEVM(msg, gp, statedb,
blockNumber, blockHash, context.Time, tx, usedGas, evm)
if  err != nil {
return  nil, fmt.Errorf("could not apply tx %d: %w", i, err)
}
```

```
receipts = append(receipts, receipt)
allLogs = append(allLogs, receipt.Logs...)
}
// 블록 보상 적용
p.chain.engine.Finalize(p.chain, header, statedb, block.Body())
return &ProcessResult{
Receipts: receipts,
Logs: allLogs,
GasUsed: *usedGas,
}, nil
}
```

3. 블록 검증 알고리즘

모든 블록은 엄격한 검증 과정을 거칩니다:

```
// ValidateBody는 블록 바디를 검증
func (v *BlockValidator) ValidateBody(block *types.Block) error {
// 이미 알려진 블록인지 확인
if  v.bc.HasBlockAndState(block.Hash(), block.NumberU64()) {
return  ErrKnownBlock
}
// 부모 블록 존재 확인
if !v.bc.HasBlockAndState(block.ParentHash(), block.NumberU64()-1) {
```

```
return consensus.ErrUnknownAncestor
}
// 엉클 검증
if err := v.bc.engine.VerifyUncles(v.bc, block); err != nil {
return err
}
// 트랜잭션 루트 검증
if hash := types.DeriveSha(block.Transactions(), trie.NewStackTrie(nil));
hash != header.TxHash {
return fmt.Errorf("transaction root hash mismatch")
}
// 인출 루트 검증 (Shanghai 포크 이후)
if header.WithdrawalsHash != nil {
if hash := types.DeriveSha(block.Withdrawals(), trie.NewStackTrie(nil));
hash != *header.WithdrawalsHash {
return fmt.Errorf("withdrawals root hash mismatch")
}
}
return nil
}


// ValidateState는 실행 후 상태를 검증
func (v *BlockValidator) ValidateState(block *types.Block, statedb *state.StateD
```

```
res *ProcessResult, stateless  bool) error {

header := block.Header()

// 가스 사용량 검증

if  block.GasUsed() != res.GasUsed {

return  fmt.Errorf("invalid gas used (remote: %d local: %d)",

block.GasUsed(), res.GasUsed)

}

// 블룸 필터 검증

rbloom := types.MergeBloom(res.Receipts)

if  rbloom != header.Bloom {

return  fmt.Errorf("invalid bloom")

}

// 영수증 루트 검증

receiptSha := types.DeriveSha(res.Receipts, trie.NewStackTrie(nil))

if  receiptSha != header.ReceiptHash {

return  fmt.Errorf("invalid receipt root hash")

}

// 상태 루트 검증

if  root := statedb.IntermediateRoot(v.config.IsEIP158(header.Number));

header.Root != root {

return  fmt.Errorf("invalid merkle root")

}

return  nil
```

```
}
```

4. 상태 저장 및 가비지 컬렉션

```
// writeBlockWithState는 블록과 상태를 데이터베이스에 저장
func (bc *BlockChain) writeBlockWithState(block *types.Block, receipts []*types.
statedb *state.StateDB) error {
// 블록 데이터 저장
blockBatch := bc.db.NewBatch()
rawdb.WriteBlock(blockBatch, block)
rawdb.WriteReceipts(blockBatch, block.Hash(), block.NumberU64(), receipts)
rawdb.WritePreimages(blockBatch, statedb.Preimages())
if err := blockBatch.Write(); err != nil {
return err
}
// 상태 커밋
root, err := statedb.Commit(block.NumberU64(),
bc.chainConfig.IsEIP158(block.Number()),
bc.chainConfig.IsCancun(block.Number(), block.Time()))
if err != nil {
return err
}
// 가비지 컬렉션 관리
if bc.triedb.Scheme() == rawdb.HashScheme {
```

```
bc.triedb.Reference(root, common.Hash{})

bc.triegc.Push(root, -int64(block.NumberU64()))

// 메모리 사용량 확인 및 정리

current := block.NumberU64()

if current > state.TriesInMemory {

chosen := current - state.TriesInMemory

if header := bc.GetHeaderByNumber(chosen); header != nil {

bc.triedb.Commit(header.Root, true)

bc.lastWrite = chosen

}

}

}

return nil

}
```

코드 구조 및 모듈 관계

```
core/

├── blockchain.go # 블록체인 메인 로직

├── state_processor.go # 트랜잭션 실행 엔진

├── block_validator.go # 블록 검증 로직

├── chain_makers.go # 체인 생성 유틸리티

├── genesis.go # 제네시스 블록 처리
```

```
├── headerchain.go # 헤더 체인 관리
└── state/ # 상태 관리 모듈
├── statedb.go # 상태 데이터베이스
├── state_object.go # 계정 상태 객체
└── journal.go # 상태 변경 저널
```

모듈 간 의존성:

- `blockchain.go` → 전체 블록체인 관리의 중심
- `state_processor.go` → 트랜잭션 실행 담당
- `block_validator.go` → 블록 검증 담당
- `state/` → 상태 관리 및 저장

실행 흐름

단계별 실행 과정

1단계: 동기화 시작

```go
func (d *Downloader) synchronise(id  string, head common.Hash, td *big.Int,
mode SyncMode) error {
// Full Sync 모드 설정
if  mode == ethconfig.FullSync {
d.mode.Store(uint32(ethconfig.FullSync))
}
// 피어 검증 및 동기화 시작
```

```
p := d.peers.Peer(id)
return d.syncWithPeer(p, head, td)
}
```

2단계: 블록 다운로드

```
func (d *Downloader) fetchBlocks(from uint64) error {
// 헤더 먼저 다운로드
headers, err := d.fetchHeaders(from)
if err != nil {
return err
}
// 각 헤더에 대해 블록 바디 다운로드
for _, header := range headers {
body, err := d.fetchBlockBody(header)
if err != nil {
return err
}
// 완전한 블록 구성
block := types.NewBlockWithHeader(header).WithBody(body.Transactions, body.Uncle
// 블록 체인에 삽입
if _, err := d.blockchain.InsertChain([]*types.Block{block}); err != nil {
return err
```

```
}
}
return nil
}
```

3단계: 블록 검증 및 실행

```
func (bc *BlockChain) processBlock(block *types.Block) error {
// 1. 블록 바디 검증
if err := bc.validator.ValidateBody(block); err != nil {
return err
}
// 2. 부모 상태 로드
parent := bc.GetBlock(block.ParentHash(), block.NumberU64()-1)
statedb, err := bc.StateAt(parent.Root())
if err != nil {
return err
}
// 3. 트랜잭션 실행
result, err := bc.processor.Process(block, statedb, bc.vmConfig)
if err != nil {
return err
}
// 4. 실행 결과 검증
```

```
if err := bc.validator.ValidateState(block, statedb, result, false); err != ni
return err
}
// 5. 상태 저장
return bc.writeBlockWithState(block, result.Receipts, statedb)
}
```

4단계: 체인 재구성 처리

```
func (bc *BlockChain) reorg(oldBlock, newBlock *types.Header) error {
// 공통 조상 찾기
var (
newChain types.Blocks
oldChain types.Blocks
commonBlock *types.Block
)
// 분기점까지 역추적
for {
if oldBlock.Number.Uint64() > newBlock.Number.Uint64() {
oldChain = append(oldChain, bc.GetBlock(oldBlock.Hash(), oldBlock.Number.Uint64(
oldBlock = bc.GetHeader(oldBlock.ParentHash, oldBlock.Number.Uint64()-1)
} else if newBlock.Number.Uint64() > oldBlock.Number.Uint64() {
newChain = append(newChain, bc.GetBlock(newBlock.Hash(), newBlock.Number.Uint64(
```

```
newBlock = bc.GetHeader(newBlock.ParentHash, newBlock.Number.Uint64()-1)

} else {

if  oldBlock.Hash() == newBlock.Hash() {

commonBlock = bc.GetBlock(oldBlock.Hash(), oldBlock.Number.Uint64())

break

}

oldChain = append(oldChain, bc.GetBlock(oldBlock.Hash(), oldBlock.Number.Uint64(

newChain = append(newChain, bc.GetBlock(newBlock.Hash(), newBlock.Number.Uint64(

oldBlock = bc.GetHeader(oldBlock.ParentHash, oldBlock.Number.Uint64()-1)

newBlock = bc.GetHeader(newBlock.ParentHash, newBlock.Number.Uint64()-1)

}

}

// 상태 롤백 및 새 체인 적용

if  err := bc.rollbackState(commonBlock, oldChain);  err != nil {

return  err

}

return  bc.applyNewChain(newChain)

}
```

상태 변화 추적

Full Sync 과정에서의 주요 상태 변화:

1. **Genesis → Syncing**: 제네시스 블록부터 동기화 시작
2. **Block Download**: 순차적 블록 다운로드

3. **Block Validation**: 각 블록의 구조적 검증
4. **Transaction Execution**: 모든 트랜잭션 실행
5. **State Validation**: 실행 결과 검증
6. **State Commit**: 상태 변경 사항 저장
7. **Chain Update**: 체인 헤드 업데이트

에러 처리 메커니즘

주요 에러 타입

```
var (
ErrKnownBlock = errors.New("block already known")
ErrUnknownAncestor = errors.New("unknown ancestor")
ErrPrunedAncestor = errors.New("pruned ancestor")
ErrInvalidNumber = errors.New("invalid block number")
ErrInvalidPoW = errors.New("invalid proof-of-work")
ErrInvalidMerkleRoot = errors.New("invalid merkle root")
ErrInvalidReceiptRoot = errors.New("invalid receipt root")
ErrInvalidGasUsed = errors.New("invalid gas used")
ErrInvalidBloom = errors.New("invalid bloom filter")
)
```

에러 복구 전략

1. **블록 검증 실패**: 해당 블록 거부 및 피어 평가 하향 조정
2. **트랜잭션 실행 실패**: 블록 전체 거부

3. **상태 불일치**: 체인 재구성 또는 동기화 재시작

4. **데이터베이스 오류**: 트랜잭션 롤백 및 재시도

성능 최적화

적용된 최적화 기법

1. 캐싱 시스템

```
const (
bodyCacheLimit = 256  // 블록 바디 캐시 크기
blockCacheLimit = 256  // 블록 캐시 크기
receiptsCacheLimit = 32  // 영수증 캐시 크기
txLookupCacheLimit = 1024  // 트랜잭션 룩업 캐시 크기
)

// 블록 캐시 초기화
bc.blockCache = lru.NewCache[common.Hash, *types.Block](blockCacheLimit)
bc.receiptCache = lru.NewCache[common.Hash, []*types.Receipt](receiptsCacheLimit
```

2. 배치 처리

```
// 여러 블록을 배치로 처리
func (bc *BlockChain) InsertChain(chain types.Blocks) (int, error) {
// 배치 크기 최적화
```

```
const maxBatchSize = 256
for i := 0; i < len(chain); i += maxBatchSize {
end := i + maxBatchSize
if end > len(chain) {
end = len(chain)
}
batch := chain[i:end]
if _, err := bc.insertChainBatch(batch); err != nil {
return i, err
}
}
return len(chain), nil
}
```

3. 병렬 검증

```
// 블록 검증을 병렬로 수행
func (bc *BlockChain) parallelValidation(blocks types.Blocks) error {
var wg sync.WaitGroup
errCh := make(chan error, len(blocks))
for _, block := range blocks {
wg.Add(1)
go func(b *types.Block) {
```

```
defer wg.Done()
if err := bc.validator.ValidateBody(b); err != nil {
errCh <- err
}
}(block)
}
wg.Wait()
close(errCh)
for err := range errCh {
if err != nil {
return err
}
}
return nil
}
```

4. 메모리 관리

```
// 트라이 가비지 컬렉션
func (bc *BlockChain) manageTrieMemory(block *types.Block) {
current := block.NumberU64()
// 메모리에 유지할 트라이 수 제한
if current > state.TriesInMemory {
chosen := current - state.TriesInMemory
```

```
// 오래된 트라이 커밋
if header := bc.GetHeaderByNumber(chosen); header != nil {
bc.triedb.Commit(header.Root, true)
}
// 참조 해제
for !bc.triegc.Empty() {
root, number := bc.triegc.Pop()
if uint64(-number) > chosen {
bc.triegc.Push(root, number)
break
}
bc.triedb.Dereference(root)
}
}
}
```

병목 지점 분석

1. 트랜잭션 실행 병목

- **문제**: 복잡한 스마트 컨트랙트 실행으로 인한 지연
- **해결**: EVM 최적화 및 병렬 실행 (제한적)

2. 디스크 I/O 병목

- **문제**: 상태 데이터 읽기/쓰기로 인한 지연

- **해결**: SSD 사용 권장 및 배치 쓰기

3. 메모리 병목

- **문제**: 대량의 상태 데이터로 인한 메모리 부족
- **해결**: 트라이 가비지 컬렉션 및 캐시 최적화

튜닝 가능한 매개변수

캐시 관련 매개변수

```
// 조정 가능한 캐시 크기
var (
TrieCleanCache = 256  // 클린 트라이 캐시 (MB)
TrieDirtyCache = 256  // 더티 트라이 캐시 (MB)
SnapshotCache = 128  // 스냅샷 캐시 (MB)
DatabaseCache = 512  // 데이터베이스 캐시 (MB)
)
```

성능 튜닝 가이드

1. **고성능 환경**: 캐시 크기 증가 (1GB+)
2. **저사양 환경**: 캐시 크기 감소 (256MB 이하)
3. **SSD 환경**: 디스크 캐시 감소, 메모리 캐시 증가
4. **HDD 환경**: 디스크 캐시 증가, 배치 크기 증가

실제 사용 예시

코드 예제

Full Sync 시작

```
// Full Sync 모드로 노드 시작
func startFullSync() error {
// 노드 설정
config := &ethconfig.Config{
SyncMode: ethconfig.FullSync,
DatabaseCache: 1024,
TrieCleanCache: 256,
TrieDirtyCache: 256,
}
// 블록체인 초기화
blockchain, err := core.NewBlockChain(db, config, genesis, nil, engine, vmConfig
if err != nil {
return err
}
// 동기화 시작
downloader := downloader.New(stateDb, mux, blockchain, nil, nil)
return downloader.Synchronise("peer-id", head, td, ethconfig.FullSync)
}
```

진행 상황 모니터링

```
// Full Sync 진행 상황 모니터링
func monitorFullSync(blockchain *core.BlockChain) {
ticker := time.NewTicker(30 * time.Second)
defer ticker.Stop()
for {
select {
case <-ticker.C:
current := blockchain.CurrentBlock()
fmt.Printf("현재 블록: %d\n", current.Number.Uint64())
fmt.Printf("블록 해시: %s\n", current.Hash().Hex())
fmt.Printf("가스 사용량: %d\n", current.GasUsed)
fmt.Printf("트랜잭션 수: %d\n", len(blockchain.GetBlockByHash(current.Hash()).Tra
// 동기화 완료 확인 (예시)
if current.Number.Uint64() >= targetBlockNumber {
fmt.Println("Full Sync 완료!")
return
}
}
}
}
```

설정 방법

geth 명령행 옵션

```
# Full Sync 모드로 시작
geth  --syncmode  full

# 캐시 크기 조정
geth  --syncmode  full  --cache  2048

# 가비지 컬렉션 조정
geth  --syncmode  full  --cache.gc  25

# 아카이브 모드 (모든 상태 보존)
geth  --syncmode  full  --gcmode  archive
```

프로그래밍 방식 설정

```
// Full Sync 설정
config := &ethconfig.Config{
SyncMode: ethconfig.FullSync,
DatabaseCache: 2048,
TrieCleanCache: 512,
TrieDirtyCache: 512,
SnapshotCache: 256,
// 아카이브 모드 설정
```

```
NoPruning: true,
ArchiveMode: true,
}


// VM 설정
vmConfig := vm.Config{
EnablePreimageRecording: true,
}
```

문제 해결 가이드

일반적인 문제와 해결 방법

1. **동기화 속도 저하**

- 원인: 디스크 I/O 병목 또는 메모리 부족
- 해결: SSD 사용, 메모리 증설, 캐시 크기 조정

2. **메모리 부족 오류**

- 원인: 트라이 캐시 크기가 너무 큼
- 해결: `--cache.trie` 옵션으로 캐시 크기 감소

3. **디스크 공간 부족**

- 원인: 전체 블록체인 데이터 저장
- 해결: 충분한 디스크 공간 확보 (1TB+ 권장)

4. **블록 검증 실패**

- 원인: 손상된 블록 데이터 또는 네트워크 오류

- 해결: 다른 피어에서 재다운로드

로그 분석

```
// Full Sync 관련 로그 모니터링
log.Info("Full sync started", "genesis", genesis.Hash())
log.Info("Block imported", "number", block.Number(), "hash", block.Hash(),
"txs", len(block.Transactions()), "gas", block.GasUsed(),
"elapsed", time.Since(start))
log.Info("State committed", "root", root, "number", block.Number())
log.Warn("Reorg detected", "oldhead", oldHead.Hash(), "newhead", newHead.Hash())
```

관련 시스템

의존성 관계

상위 의존성

- **eth/handler**: 블록 수신 및 처리 조정
- **eth/downloader**: 블록 다운로드 관리
- **miner**: 마이닝 및 블록 생성

하위 의존성

- **core/state**: 상태 관리 및 트라이 조작
- **core/vm**: EVM 실행 엔진
- **consensus**: 합의 알고리즘 구현

- **ethdb**: 데이터베이스 접근 계층

상호작용 패턴

상태 관리와의 상호작용

```
// 상태 데이터베이스 생성
statedb, err := state.New(parentRoot, bc.stateCache, bc.snaps)
if err != nil {
return err
}

// 트랜잭션 실행으로 상태 변경
receipt, err := ApplyTransaction(bc.chainConfig, bc, nil, gp, statedb,
header, tx, usedGas, vmConfig)

// 상태 커밋
root, err := statedb.Commit(header.Number.Uint64(),
bc.chainConfig.IsEIP158(header.Number))
```

EVM과의 상호작용

```
// EVM 인스턴스 생성
context := NewEVMBlockContext(header, bc, nil)
evm := vm.NewEVM(context, statedb, bc.chainConfig, vmConfig)
```

```
// 트랜잭션 실행
result, err := ApplyMessage(evm, msg, gp)
if err != nil {
return nil, err
}
```

합의 엔진과의 상호작용

```
// 블록 검증
if err := bc.engine.VerifyHeader(bc, header, true); err != nil {
return err
}

// 블록 완료 처리 (보상 지급 등)
bc.engine.Finalize(bc, header, statedb, block.Body())
```

영향 범위

직접적 영향

- **보안 수준**: 최고 수준의 검증으로 네트워크 보안 기여
- **동기화 시간**: 가장 오래 걸리는 동기화 방식
- **리소스 사용량**: CPU, 메모리, 디스크 모두 높은 사용량

간접적 영향

- **네트워크 분산화**: 완전한 노드로서 네트워크 건강성 기여
- **히스토리 보존**: 전체 블록체인 히스토리 보존
- **개발자 도구**: 완전한 상태 접근으로 디버깅 및 분석 지원

Full Sync의 특징과 장단점

장점

1. **완전한 검증**: 모든 트랜잭션을 직접 실행하여 검증
2. **최고 보안**: 제3자 신뢰 없이 독립적 검증
3. **완전한 히스토리**: 모든 블록과 상태 변화 추적 가능
4. **네트워크 기여**: 완전한 노드로서 네트워크에 최대 기여

단점

1. **긴 동기화 시간**: 수일에서 수주 소요
2. **높은 리소스 요구**: 대용량 디스크, 메모리, CPU 필요
3. **높은 대역폭 사용**: 모든 블록 데이터 다운로드
4. **유지 비용**: 지속적인 하드웨어 업그레이드 필요

사용 사례

1. **아카이브 노드**: 전체 히스토리 보존이 필요한 경우
2. **연구 및 분석**: 블록체인 데이터 분석 및 연구
3. **최고 보안**: 최대한의 검증이 필요한 금융 서비스
4. **개발 및 디버깅**: 완전한 상태 접근이 필요한 개발 환경

Full Sync는 Ethereum의 가장 보수적이고 완전한 동기화 방식으로, 최고 수준의 보안과 검증을 제공합니다. 비록 시간과 리소스가 많이 소요되지만, 네트워크의 분산화와 보안에 핵심적인 역할을 담당하

는 중요한 동기화 메커니즘입니다.